Core Explorer Lite: Design, Implementation, and Evaluation of a Voxel-Based Platform Game

Jacky Hui (Member A) Jerry Xie (Member B)

COMP3421/COMP9415 Computer Graphics
UNSW Sydney, Term 3, 2026

12 November 2026

Abstract

This report presents the design and evaluation of Core Explorer Lite, a third-person voxel platform game implemented in C++17 and OpenGL 3.3 Core. A 96^3 voxel representation supports an untimed training environment and two missions with different navigation and movement demands. The implementation combines hierarchical robot and gate models, world-space Blinn–Phong lighting, procedural normal maps and a bounded event-driven particle system. Hidden-face removal is evaluated against a baseline that submits all voxel faces. At a 1920×1080 framebuffer resolution, with 60 warm-up frames and 240 measured frames per run, the optimisation reduces mean scene-rendering time by 8.1–19.7% across the three configurations on an RTX 4060 Laptop GPU. The results demonstrate a reduction in geometry cost without changing draw-call counts. Remaining map/test inconsistencies and unverified VLAB/CSE execution limit conclusions about end-to-end gameplay completeness and platform portability.

Keywords: voxel rendering; hierarchical modelling; normal mapping; particle systems; performance evaluation.

1. Introduction and Game Design

Core Explorer Lite is a single-player, third-person voxel platform game. A repair robot collects every energy core in a facility and reaches the unlocked exit. Its three authored environments combine spatial navigation, timed hazards and physical interactions. The training yard introduces the mechanisms; the two missions place different demands on route planning and movement precision.

These are horizontal room dimensions, not the storage volume: every configuration uses a $96 \times 96 \times 96$ voxel world, including empty cells. Table 1 describes the evaluated level files. The two missions are shown in Figure 1.

1.1 Controls and rules

WASD moves relative to the camera; **mouse** orbits and the **wheel** changes distance. **Shift** sprints, **Space** jumps, **C** toggles crouching and **E** interacts with cores, relays and portal endpoints. Checkpoints save automatically when reached. **R** restarts, **Esc** pauses, and **V** toggles a north-up orthographic surveillance view with map-relative movement. Bindings are configurable in Settings. The minimap supports navigation in the follow view.

The robot has five armour units. Falling onto the void floor costs one armour unit and returns the robot to its checkpoint. Damage has a cooldown; loss of all armour triggers a 2.2-second reboot at the last checkpoint or entry. Timed missions allow three lives; ordinary reboots retain collected cores and elapsed time, restore armour and grant one second of protection. Exhausting lives restarts the mission; reaching the time limit fails the run. Training has unlimited time and lives. Relays temporarily disable linked shock plates, pressure plates unlock doors, and jump pads launch the robot automatically. Pauses and level-transition holds stop the gameplay clock.

2. System Architecture

2.1 World, state and collision

`LevelData` parses text files into material-valued cells and typed mechanisms; `Game` owns the voxel world, player, timers, hazards, interaction state and particles. The main loop separates title/play/pause/settings screens and level transitions from the simulation states Playing/Won/Lost. A reset reconstructs the chosen configuration and clears transient state. Simulation advances in substeps of at most $1/120$ second. A vertical player cylinder is tested against occupied voxel boxes, moving-hazard parts and mechanism fixtures; movement is resolved by axis, with crouch clearance checked before standing. Render-only limb motion does not change the collision body.

2.2 Geometry and rendering

C++17 and OpenGL 3.3 Core render true 3D geometry with a depth buffer. Terrain is generated as UV-mapped triangle faces; shared boxes, spheres and cylinders form the robot and mechanisms. Model instances are grouped by primitive and blend mode. `Renderer::prepare` builds/uploads geometry and instances; `draw` applies the camera, GLSL materials and scene effects. Framebuffer dimensions drive the viewport and projection. The procedural star background uses a fullscreen triangle reconstructed from viewing directions; it uses no sky image. F1 displays live geometry and camera diagnostics (Figure 2).

2.3 Two multipart hierarchies

For node i, the inherited joint matrix is $J_i = J_{p(i)} T_i R_{x,i} R_{y,i} R_{z,i}$; a root uses the object's world transform. Its drawn mesh uses $M_i = J_i S_i$. Keeping mesh dimensions out of the inherited joint transform prevents a scaled torso from stretching the arms. The robot's root handles world motion/yaw, while children supply gait and reaction poses. The exit gate has a separate root and independently translating panels (Figure 3); handles and stripes follow their parent panel.

Table 1. Authored configurations and gameplay objectives. Dimensions denote horizontal room extents.

Configuration	Current layout	Objective and distinguishing mechanics
Tutorial Yard	32×32, 3 cores	Untimed; crawl passage, relay-controlled shock floor, moving obstacle, ground/upper doors, jump pad and portal pair.
Helios Yard	84×60, 4 cores	600 seconds; four districts with rooftop access, electric-floor traversal and layered/corridor mazes; five pressure-plate doors and one portal pair.
Corona Run	76×76, 9 cores	300 seconds; elevated parkour above a hazardous red floor, narrow beams, gaps and moving obstacles; four checkpoints and four jump pads, without portals.



(a) Helios Yard: districts and mechanisms

(b) Corona Run: elevated routes over the void

Figure 1. Current mission configurations, captured in the whole-room overview. The HUD shows armour, remaining lives, time and the required core slots.



Figure 2. Perspective scene with live diagnostics: 71,781 submitted triangles, eight draw calls, 1,924 solid voxels and 3,726 exposed faces. Camera position is $(22.73, 6.10, 38.41)$ with a 58° vertical field of view. The screenshot is cropped to emphasise scene geometry and the diagnostic overlay.

3. Surface Shading and Normal Mapping

3.1 Lighting model and material differentiation

The shader uses world-space positions and view/light directions. Geometric normals are transformed by the inverse transpose of the model matrix. The opaque surface colour combines hemispheric ambient light, Lambert diffuse, Blinn–Phong specular, emission and distance fog:

$$\begin{aligned} C &= a\big(A + L_d \max(0, n\cdot l)\big) \\ &\quad + L_s k_s \max(0, n\cdot h)^p + E, \\ h &= \frac{l + v}{\|l + v\|}. \end{aligned} \tag{1}$$

The implementation generates five periodic 64×64 albedo/normal texture layers. Materials differ in surface pattern, gloss and reflectance rather than colour alone. Textures repeat, use linear sampling and mipmaps. Normal data is stored as linear RGB. The lighting formulation is Blinn–Phong rather than a microfacet BRDF; shadow mapping is outside the implemented scope.

Table 2. Blinn–Phong material parameters.

Material	Exponent p	Specular k_s
Ceramic	80	.25
Tread steel	24	.20
Brushed alloy	110	.65
Rubber	8	.025
Painted casing	48	.18

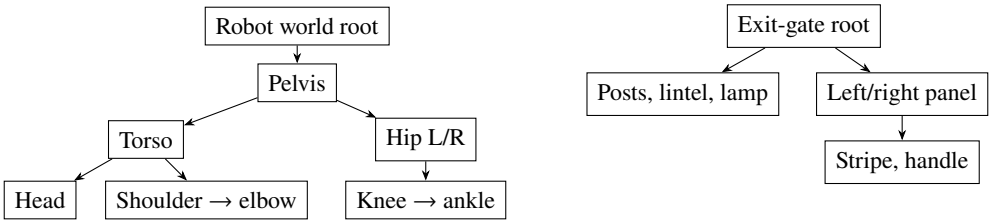


Figure 3. Simplified parent-child graph of the two textured articulated models. Each left/right branch is instantiated separately.

3.2 Tangent-Space Normal Mapping

Procedural height-field differences produce a tangent-space normal for each texel. In the fragment shader, derivatives of world position and UV coordinates recover the local UV basis. The tangent is orthogonalised against the geometric normal, and the bitangent sign follows the UV orientation; this preserves mirrored-chart handedness. A mesh tangent provides a fallback when the derivative basis is degenerate. The sampled normal is decoded from $[0, 1]$ to $[-1, 1]$, its lateral components are scaled by material relief and the interactive strength, and the result is normalised and transformed by TBN into world space before lighting. Figure 4 compares the resulting surface response.

3.3 Demonstration and validation

F2 toggles mapping, **F3** compares OFF/ON across the screen, **F4** freezes gameplay, and **F5** cycles strength through 0, 0.5, 1, 1.5 and 2. The zero-strength shader branch uses the same geometric normal as mapping OFF. No extra vertices are added: surface lighting changes without changing terrain topology or collision.

The paired views can be reproduced from the repository root using `--play`, `--freeze`, `--frames 2` and `--capture on.png`, then add `--normal-off` for the other view. Inspection of sloped and curved model parts complements terrain-face comparisons, since basis errors may remain hidden on an axis-aligned floor. The current evidence establishes the visible material response; it does not establish a numerical image-equivalence result for the interactive capture path.

4. Particle Effects and Procedural Animation

4.1 Bounded Particle Lifecycle

`Game` owns a reusable pool of 128 particles, each with position, velocity, size, colour, type and remaining lifetime. An emitter fills inactive slots and skips excess requests when the pool is full. Core collection emits 20 cyan sparks, damage emits 24 amber sparks/debris, and a jump pad emits 20 upward rays followed by three trail particles every 0.04 seconds during the short ascent window. Footfalls and landings also produce dust. These effects distinguish successful interactions, harm and assisted movement (Figure 5).

During simulation substeps, particles lose lifetime and integrate gravity and exponential drag. Axis-wise collision checks against terrain and scene fixtures damp/reflect motion. Expired slots become available for reuse; a level reset clears the pool. Pausing/freezing stops simulation; after a win/loss, remaining effects can settle while the gameplay timer stays stopped. Opaque debris is drawn with the scene; soft sparks/rays are grouped into instanced passes, depth-tested with depth writes disabled and alpha/additive blending. The opaque pass precedes these soft effects.

4.2 Purposeful hierarchical animation

`CourierPose` poses the skeleton from movement and interaction state. Gait phases coordinate hip, knee and ankle rotation, with arm swing and torso motion; sprinting changes cadence and stride. Crouching lowers the pose, while jump, impact and shutdown states provide distinct feedback. Squash/stretch and body hop are visual offsets around the feet. The exit gate translates its two child panels when all cores are collected. Local animation therefore remains independent of the object's root position and orientation; it is procedural animation rather than a rigid-body ragdoll (Figure 6).

4.3 Validation boundary

The event captures completed without reported OpenGL errors and showed the expected particle counts; the impact capture also reduced armour from five to four. Tests cover movement, damage cooldown, particle motion/recycling, restart state and pad emission. However, the gameplay test executable stops at a later Helios layout assertion, and route validation is incomplete (Section 5). These observations support specific mechanisms, not a claim that every mission can currently be completed.

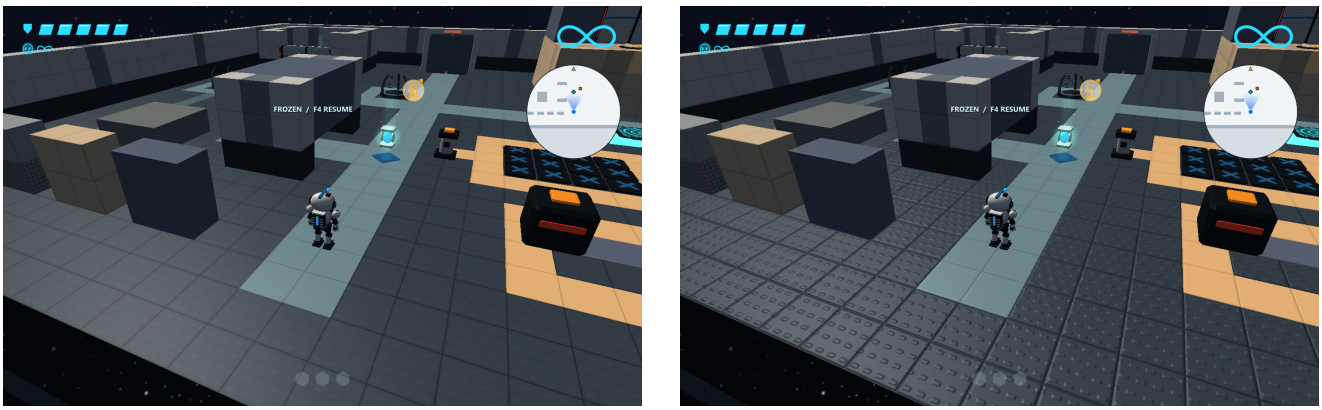
5. Experimental Evaluation

5.1 Controlled comparison

The core optimisation removes a voxel face when its neighbouring cell is occupied. The baseline emits all six faces of every solid voxel. Both modes use the same instance grouping, shaders and effects; hidden-face removal is the only changed variable and is not counted again as an acceleration challenge. F6 toggles this optimisation interactively. Static terrain is rebuilt when the world revision or mode changes, outside the timed boundary.

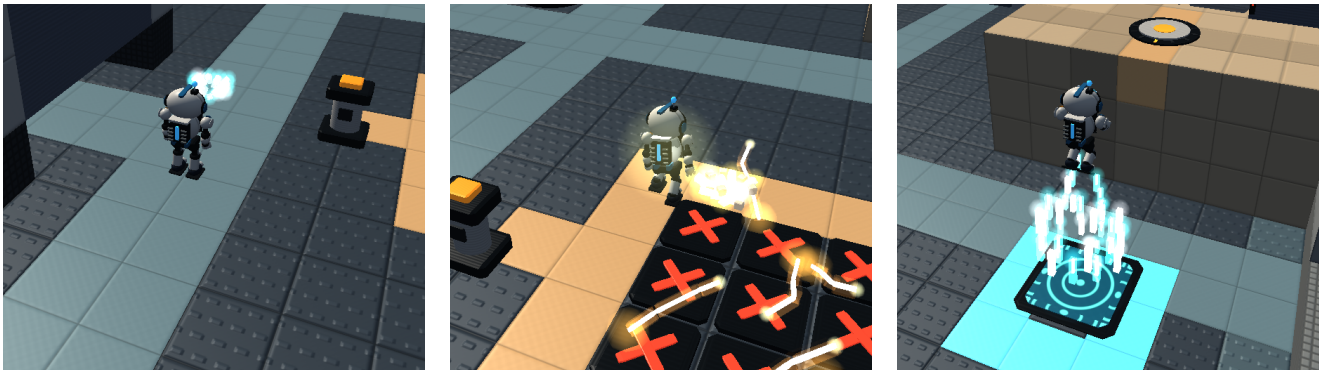
Measurements were collected on 12 November using a Windows Release build (GCC 16.2.0), NVIDIA GeForce RTX 4060 Laptop GPU, driver 621.01, OpenGL 3.3. Each run used a **1920×1080 framebuffer**, VSync off, **60 unmeasured warm-up frames** and **exactly 240 measured scene-render frames**. Each mode was repeated three times, alternating order. Table 3 averages those three 240-frame arithmetic means; every row has equal weight.

All scenes use the 96^3 world and their fixed authored level files (no layout generation seed). The player is placed on the first jump pad, simulated for $24 \times 1/120 = 0.2$ seconds, then frozen with 35 active particles. The camera is at $(20, 17, 38)$, looking at $(20, 1, 19)$ with up $(0, 1, 0)$; vertical FOV is 50°, near/far planes 0.1/300. Normal mapping is ON at strength 1; the procedural sky and all prepared scene effects are included with render time fixed at zero. The same camera is retained



(a) Normal mapping OFF (b) Normal mapping ON, strength 1.0

Figure 4. Same tutorial spawn, camera and frozen gameplay. Tread relief changes the floor highlights while the geometric silhouette remains unchanged. Sky/UI animation uses wall-clock time, so these separate captures are visual evidence rather than a pixel-identical whole-image test.



(a) Core: 20 live particles **(b) Impact: 24 live particles** **(c) Pad: 35 live particles**

Figure 5. Detail crops from actual event captures (`--effect core`, `impact`, `pad`); counts are the capture logs, not the maximum pool size. The pad count includes already-emitted trail particles.



Standing **Running** **Pad ascent**

Figure 6. Contrasting robot poses, enlarged from gameplay captures. Different joint angles, body height and limb placement make locomotion state visible.

across modes and levels; it samples a fixed view, not an entire play-through.

Interpretation. Triangle counts decrease by 16.8%, 41.4% and 47.0%, respectively. Draw-call counts are unchanged because culling reduces the terrain mesh, not the number of material/instance batches. Timing gains are smaller than geometry reductions: sky shading, other fragments and fixed draw overhead remain. Across repeats, baseline/optimised ranges were 2.549–2.635/2.369–2.426 ms (Tutorial), 2.950–3.154/2.467–2.475 ms (Helios), and 2.851–3.080/2.339–2.377 ms (Corona). These are local observations, not cross-device guarantees.

Timing boundary and image checks. `glFinish` drains previous work before the CPU timer and waits for scene completion before stopping it. Thus the result includes scene submission and completion, not a pure GPU timestamp. Loading, mesh preparation, simulation, ImGui, screenshot capture and buffer swapping are excluded. First-pair baseline/optimised screenshots differed in fewer than 0.005% of pixels, with mean absolute RGB error below 0.002 on the 0–255 scale; they are nearly, but not exactly, identical. Exact CSVs and source fingerprints accompany the editable report.

Reproduction. The benchmark is invoked from the repository root using `--benchmark`, `--level 0` and

Table 3. Current-snapshot scene-rendering results. Reduction is $100(1 - t_{opt}/t_{base})$. Counts include terrain, model instances, effects and the sky triangle.

	Mean ms/frame				
Scene	**Base**	**Opt.**	**Reduction**	**Triangles: base → opt.**	**Draws**
Tutorial	2.606	2.395	8.1%	93,129 → 77,493	9 → 9
Helios	3.019	2.470	18.2%	380,137 → 222,713	10 → 10
Corona	2.929	2.353	19.7%	321,017 → 170,173	10 → 10

`--benchmark-output tutorial.csv`. Levels 1 and 2 select Helios and Corona, respectively. The executable performs all six measurements per level and saves the first baseline/optimised image pair. Use the current source and level files: the November 7 binary and older benchmark tables describe different scenes.

5.2 Current validation status

The Windows game and five game-test targets build successfully. Of six registered CTest suites, tutorial and surveillance-camera suites pass. The editor and Corona suites expect ten Corona cores (now nine), and the Corona suite also expects at least eight movers (now six); the gameplay suite expects six Helios doors/plates (now five); the Helios route search cannot reach the indexed west inner plate. This leaves three game suites plus the editor suite failing. These failures require map/test reconciliation and route revalidation. A successful VLAB/CSE build and play-through has not yet been established for this snapshot.

6. Contributions and Discussion

6.1 Responsibilities and integration

Jacky Hui is Member A, and Jerry Xie is Member B. Their responsibilities follow the division established in the revised project plan. **Member A (Jacky Hui)** covers player movement and interactions, hazards, lives/checkpoints, training, controls and menus, robot animation, gameplay feedback and particle lifecycle. **Member B (Jerry Xie)** covers voxel terrain and level layouts, materials and lighting, normal mapping, environment rendering, optimisation measurement and the planned VLAB/CSE validation. Both roles meet at mechanism placement, scene state, camera behaviour, regression testing and the demonstration.

Relevant history includes `53a3ba9` (jump-pad plume/trail), `8ae7ec6` (five procedural materials), `8909214` (normal/particle demonstrations and benchmarks), and `1af555e` (tutorial, Corona and reboot integration). Jerry's `735305f` adds the minimap and mouse-linked facing; `e71999d` documents its integration. These commits locate feature/integration evidence; the responsibility split is not a claim that each commit has the corresponding member as its sole author. Measurements include local changes after `1af555e`.

6.2 Individual technical reflections

Member A (Jacky Hui). The main lesson in the gameplay role is to separate presentation from authoritative simulation state. Limb animation and recoil should communicate movement without moving the collision body through walls. Death, reboot, pause and full restart also need different reset rules: retaining cores during recovery must not retain them after a new run. Event captures help inspect this feedback, while state-based tests are needed to check timers, cooldowns and particle cleanup. The remaining route failures show why visible local mechanisms are insufficient evidence of a complete playable mission.

Member B (Jerry Xie). The rendering role highlights the importance of consistent coordinate spaces and controlled comparisons. A normal map needs a correct UV basis, especially on scaled and curved parts; extra texture detail alone does not establish correctness. Hidden-face removal substantially reduces submitted triangles, but scene time also depends on shader and fixed costs. Level-file edits must be reconciled with mechanism indices and route tests before freezing the final scenes. The outstanding platform check remains an essential part of this role's planned delivery.

6.3 Comparison with the Project Plan

The two selected challenges remain Normal Mapping and Particles. Lite replaces the earlier courtyard/corridor missions with Helios and Corona, grows storage from 64^3 to 96^3, reduces the particle pool from 1,024 to 128, and uses a procedural sky. Optional power-ups/speed pads from the plan are outside the current authored levels. Reboot timing and progression rules have also changed. Final-stage priorities are resolving map/test mismatches, validating complete routes on VLAB/CSE, and refreshing evidence after changes.

6.4 External Software and Assets

The project extends the course starter's window, shader and UI foundation. CMake `FetchContent` integrates GLFW for window/context management and input [1], GLAD for OpenGL function loading [2], GLM for vector and matrix mathematics [3], Dear ImGui and its GLFW/OpenGL backends for the interface [4], and stb for image input/output [5]. The evaluated build uses existing local dependency overrides; declared versions and licence sources are provided in the references.

Geometry, material textures and the current star sky are generated by project code; no external model or sky photograph is used in the reported captures. The bundled Rajdhani SemiBold font is distributed under SIL OFL 1.1 [6]. On Windows, the application may instead load Segoe UI Bold from the installed system; its font file is not redistributed with the project [7].

7. Conclusion

Core Explorer Lite integrates voxel collision, articulated modelling, material shading and event-driven feedback within three distinct configurations. The controlled benchmark shows that removing internal voxel faces reduces submitted triangles and scene-rendering time, while substantial shader and fixed rendering costs remain. Normal-map comparisons

and event captures provide evidence for the two selected challenge tasks. Complete-route validation, reconciliation of map-dependent tests and verification on VLAB/CSE remain necessary before the evaluated development version can be treated as a final submission.

References

[1] GLFW Development Team, *GLFW 3.3.10*, zlib/libpng licence. [Online]. Available: https://github.com/glfw/glfw/blob/3.3.10/LICENSE.md.

[2] D. Herberth, *GLAD 2.0.8*, MIT licence; additional Khronos terms for specifications. [Online]. Available: https://github.com/dav1dde/glad/blob/v2.0.8/LICENSE.

[3] G-Truc Creation, *OpenGL Mathematics (GLM) 1.0.3*, MIT or Happy Bunny licence. [Online]. Available: https://github.com/g-truc/glm/blob/1.0.3/copying.txt.

[4] O. Cornut, *Dear ImGui 1.92.9b*, MIT licence. [Online]. Available: https://github.com/ocornut/imgui/blob/v1.92.9b/LICENSE.txt.

[5] stb contributors, *stb*, commit `2c980bb`, MIT or public-domain alternative. [Online]. Available: https://github.com/nothings/stb/blob/2c980bb59875b0d32144a71867fbdebb2f77cd20/LICENSE.

[6] Indian Type Foundry, *Rajdhani*, SIL Open Font License 1.1, 2014. [Online]. Available: https://github.com/google/fonts/tree/main/ofl/rajdhani. Licence retained in `assets/fonts/OFL-Rajdhani.txt`.

[7] Microsoft, *Segoe UI font family*, installed Windows font; separate terms apply to redistribution. [Online]. Available: https://learn.microsoft.com/en-us/typography/font-list/segoe-ui.